Product Requirement Document (PRD)

AutoThreads-AI: Zero-Cost Production Specification

Status:	Final / Production-Ready	**Version:**	3.0
Target Engine:	GitHub Actions (Serverless)	**Date:**	July 2026

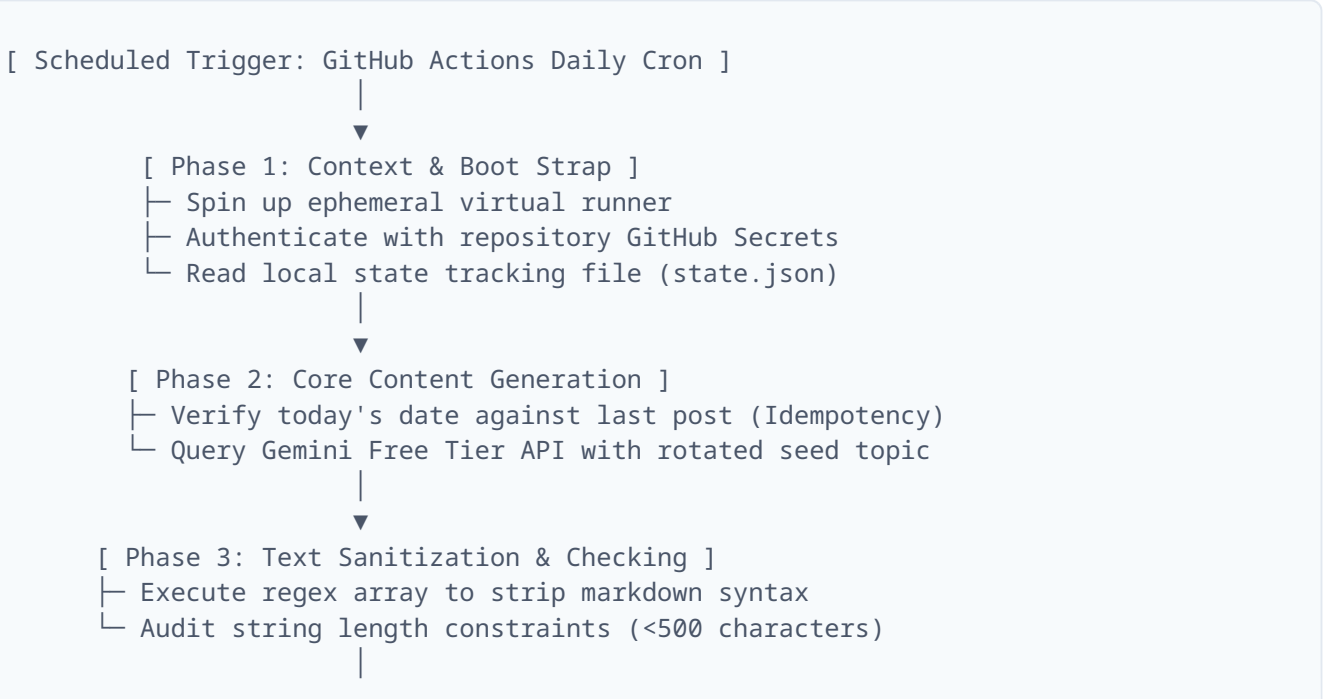
1. Objective & Scope

The objective of this project is to build an entirely autonomous, zero-cost, serverless background publishing system that curates, generates, and validates concise technical content focused on AI and emerging technology, publishing it directly to Instagram Threads at daily intervals.

The system operates strictly within free-tier resource structures. It utilizes GitHub Actions as its primary serverless compute engine, Google Gemini's Free Tier for content curation, and a Git-backed state tracking mechanism inside the repository itself to handle persistence without requiring paid external databases or servers.

2. Serverless Architectural Workflow

Because the compute runner environment is completely stateless and destroys itself after each execution, the repository uses a self-referential Git tracking loop to maintain system state, prevent duplicate posts, and handle token lifespans securely.



```
                    ▼
        [ Phase 4: Asynchronous Publishing Engine ]
        ├─ POST payload to Meta Graph API for Container ID
        ├─ Enter 5-second interval verification polling loop
        └─ Finalize transaction via Threads Publish endpoint
                    │
                    ▼
      [ Phase 5: Self-Writing State Persistence ]
      ├─ Append timestamp data and calculate token lifespan age
      ├─ If age approaching 60 days -> Dispatch external alert webhook
      ├─ If 45 days of code silence -> Trigger text keep-alive commit
      └─ Git commit and push updated state.json back to main branch
```

3. Detailed Component Specifications

3.1 Compute Setup & Scheduler Engine

- **Infrastructure Strategy:** The system executes exclusively on a virtual GitHub Actions runner using the default free Linux environment (`ubuntu-latest`).
- **Trigger Schedule Configuration:** The workflow runs automatically via an schedule cron statement. To counter the unpredictable delays inherent in the free global GitHub actions queue, the schedule targets an off-peak global traffic window. A manual trigger override (`workflow_dispatch`) is explicitly declared alongside it to allow for on-demand execution, manual debugging, and instant token synchronization.
- **Permissions:** The environment must be explicitly granted write access permissions (`contents: write`) to allow the runner to modify the repository filesystem and commit data back to itself.

3.2 Content Generation Engine (Gemini Free Tier)

- **API Utilization:** The workflow connects directly to the Google Gemini API, running entirely within the free-tier quota limits (allowing up to 15 requests per minute, easily covering a single daily run).
- **Prompt Architecture:** Gemini is initialized with strict system instructions defining its identity as a technical subject matter expert. It is instructed to write technical value-bombs that are concise, punchy, and dense.
- **Algorithmic Topic Rotation:** The system parses a static array of technological areas located inside the codebase. The engine extracts the current day of the year, running a mathematical modulo calculation against the total length of the array to select a unique topic index. This guarantees consistent, varied daily content cycles without needing an external state manager.

3.3 Validation & Content Sanitization Layer

- **Visual Optimization Filter:** Threads does not natively parse or render Markdown syntax. The text returned from the Gemini API passes through a regex filter that strips out bold markdown structures (`**`), headers (`#`), code ticks (`` ` ``), and wrapping quotation marks to preserve clean mobile rendering.
- **Defensive Bounds Checking:** Threads enforces a strict hard limit of 500 characters per post. The sanitization layer calculates the exact character length of the cleaned string. If the text measures 491

characters or more, the system blocks transmission to Meta, raises a warning flags array, and re-queries the Gemini engine using an emergency single-sentence reduction instruction.

3.4 Asynchronous Meta Threads Engine

- **Staging Phase:** The cleaned text is dispatched via a POST operation to the Meta Threads container initialization endpoint.
- **Deterministic Polling Loop Workaround:** Meta processes and reformats raw text containers asynchronously, returning an immediate status of `IN_PROGRESS`. To circumvent this, the runner enters a precise validation loop:
  - The workflow sleeps for 5 seconds using an execution block.
  - It hits the Meta media container status endpoint to check processing health.
  - If the status reads `FINISHED`, it exits the loop and moves forward.
  - The loop times out after 6 attempts. If it fails to resolve, the execution is terminated to save free runner minutes.
- **Finalization Phase:** The system passes the validated Container ID to the live publishing endpoint, pushing the content live to the user's public profile.

3.5 Self-Writing State Persistence Engine

- **Anti-Duplication (Idempotency Lock):** To protect the profile against accidental duplicate posts caused by delayed GitHub cron retries, the system opens a localized tracking file (`state.json`). If the value matching `last_post_date` equals the current system date, the workflow exits immediately.
- **Automated Keep-Alive Trigger:** GitHub automatically disables scheduled workflows in repositories that have not received manual code modifications or commits for 60 consecutive days. To defeat this limitation, the system checks the timestamp of the last manual developer commit. If the age approaches 45 days, the runner modifies a hidden tracking value inside an internal system file.
- **Database Write-Back Phase:** Following a verified, successful post or an active keep-alive update, the workflow edits the localized tracking file with updated parameters. The system automatically performs a command sequence: staging the updated tracking file, committing the changes with an automated system user tag, and pushing the new state directly back to the active main branch.

> **Critical Platform Dependency: Token Expiration Alerting**
>
> Meta's long-lived access tokens expire exactly 60 days from creation and cannot be programmatically refreshed without human login consent. The persistence engine tracks the creation timestamp recorded in the secret configuration. When the token lifespan exceeds 50 days, the engine triggers an outbound HTTP POST payload containing a refresh reminder directly to a free user communication channel (such as a Discord or Telegram webhook).

4. Environment Schema & Security Architecture

All operational tokens are entirely decoupled from the core codebase. They are encrypted and stored within GitHub Repository Secrets, which injects them securely into the environment variables during runtime:

- **GEMINI_API_KEY:** Secret access key provisioned through the Google AI Studio portal.
- **META_ACCESS_TOKEN:** The 60-day extended Graph API token required to authenticate with the Threads profile.
- **THREADS_USER_ID:** The public identifier of the target Threads account.
- **NOTIFICATION_WEBHOOK:** The target Discord, Telegram, or Slack webhook URL used to pass runtime system warnings and token expiration warnings.

5. Non-Functional Requirements & System Health

5.1 Resource Constraints & Performance Limits

The entire end-to-end workflow execution loop—encompassing repository retrieval, context compilation, content validation, network polling, and database write-backs—must complete within 90 seconds, ensuring it consumes negligible amounts of GitHub's free 2,000 monthly runner minutes. All third-party GitHub Actions blocks inside the workflow definition must be pinned to exact, immutable cryptographic SHA hashes rather than mutable version tags, shielding the autonomous ecosystem from surprise upstream updates.

5.2 System Recovery & Fault Isolation

If a connection timeout occurs while querying external APIs, the engine runs an inline retry loop using exponential intervals (retrying at 10 seconds, then 30 seconds) before failing. If an API endpoint returns a critical fatal error code, the system immediately drops out of the loop, commits the error log to the state file, triggers the webhook alert, and terminates cleanly without causing a cascading system hang.